0.0.1 Question 1a

In this project we are essentially trying to answer the question. "How much is a house worth?" - Who might be interested in an answer to this question? **Please list at least three different parties (people or organizations) and then describe whether each one has an interest in seeing the housing price to be either high or low.**

The homeowners would likely be interested in the price of there house being both high and low as if they are looking to sell or rent out the house they would like to see a high price in order to profit more. However if they aren't they wouldn't want to have a high house price as that would mean that they have to pay more taxes on the house which they likely do not like. The government also as an interest in seeing the housing prices be high as that means they are able to collect more taxes on them as a result of properity tax. Banks could also be another party as low prices might means people are getting more loans as they are trying to buy houses which is good for the bank.

0.0.2 Question 1b

- 1bi). Which of the following scenarios strike you as unfair and why? You can choose more than one. There is no single right answer, but you must explain your reasoning.
- 1bii). Would you consider some of these scenarios more (or less) fair than others? Why?

Scenario A: A homeowner whose home is assessed at a higher price than it would sell for.
Scenario B: A homeowner whose home is assessed at a lower price than it would sell for.
Scenario C: An assessment process that systematically overvalues inexpensive properties and undervalues expensive properties.
Scenario D: An assessment process that systematically undervalues inexpensive properties and overvalues expensive properties.

Write your full answers to both parts in the cell below:

1bi) All of the following scenarios are unfair as they privilege someone over someone else for no reason other than the evaluators either making a mistake or showing intentional favoritism. The homeowner whose house is higher priced could sell the house for more, but they also have to pay more taxes, which is unfair to that individual. The lower price is the inverse, where they would get less for selling but also would pay much less in taxes. The other two have the same issues; however, they are systemic, meaning they likely target specific demographics and people in certain areas, which is also unfair.

1bii) The systemic issues are more unfair as they are much less likely to be due to random chance and mean that there is a deliberate reason these people are treated unfairly due to the trend of them having worse conditions compared to someone else, just due to the area they are in or stereotypes associated with them.

0.0.3 Question 1d

- 1di). What were the central problems with the earlier property tax system in Cook County as reported by the Chicago Tribune?
- 1dii). What were the primary causes of these problems? (Note: in addition to reading the paragraph above you will need to **read the Project Case Study.pdf explaining the context and history of this dataset before answering this question).**

1di) The central problems was that the Cook County tax system result in a highly regressive tax system which disprportionawtly affected African-Amercan and Latin American homeowners by burdening them with higher taxes even though the value of there homes was lower on average compareed to other races.

1dii) One of the primary problems is the appeals system which allows for people of higher income to make a better case in order to lower their taxes. This is becasue while every can appeal they don't have the same opportunity to do so.

0.0.4 Question 2b:

Since we're trying to predict `Sale Price`, next we'll look for missing or unusual outliers in that field.

Examine the `Sale Price` column in the `training_val_data` DataFrame and answer the following questions:

- 2bi). Does the `Sale Price` data have any missing, N/A, negative or 0 values for the data? If so, propose a way to handle this.
- 2bii). Does the `Sale Price` data have any unusually large outlier values? If so, propose a cutoff to use for throwing out large outliers, and justify your reasoning).
- 2biii). Does the `Sale Price` data have any unusually small outlier values? If so, propose a cutoff to use for throwing out small outliers, and justify your reasoning.

Below are three cells. The first is a Markdown cell for you to write up your responses to all 3 parts above. The second two are code cells that are available for you to write code to explore the outliers and/or visualize the Sale Price data.

0.0.5 Question 2b i, ii, iii answer cell: *Type your responses to all three parts in this cell...*

2bi) There are no missing values in the sale price data and no zero values so they don't need to be handled

2bii) There are about 5 values which are unusually largre compared to the rest of the values and in order to handle them a cuttoff could be 1000000 or 900000 as there are likely not many houses which are over a million dollar and likely don't reflect the average valuation of most houses in this area.

2bii) There are a lot of 1 dollar values and other incrediblly low values that you can't by a house for or any other properity. After a quick look at zillow the lowest value properity was listed was 15000 dollars so that is a resonable. However looking at the datta further there are a lot under this value so a lower value like 10000 might make more sense. I would say that based on the listing and values I would use the lower cutoff and then if needed adjust this and make it higher to get a btter look at the data

```
In [12]: sale_price = tr_val_data['Sale Price']
         missing_values = sale_price.isna().sum()
         zero_values = (sale_price == 0).sum()

         print("There are", missing_values, "missing values in sale price")
         print("There are", zero_values, "zero values in sale price")
         # your code exploring Sale Price above this line
```

```
There are 0 missing values in sale price
There are 0 zero values in sale price
```

```
In [13]: value_counts = tr_val_data['Sale Price'].value_counts()
         max = tr_val_data['Sale Price'].max()
         min = tr_val_data['Sale Price'].min()

         print(value_counts)
         print("Max values in sale price:", max)
         print("Min values in sale price:", min)
         count_values_1000_1500 = tr_val_data[(tr_val_data['Sale Price'] >= 10000) & (tr_val_data['Sale
         count_values_under_1000 = tr_val_data[tr_val_data['Sale Price'] < 10000].shape[0]
         print("number of values between", count_values_1000_1500, "10000 and 15000")
         print("number of values under", count_values_under_1000, "10000")
         sorted_sale_price_head =sorted_sale_price_head = tr_val_data[tr_val_data["Sale Price"] > 1000]
         print("Lowest 5 Sale Prices:")
         print(sorted_sale_price_head)
         sorted_sale_price_tail = tr_val_data["Sale Price"].sort_values().tail(10)
         print("\nHighest 5 Sale Prices:")
         print(sorted_sale_price_tail)
         # optional extra cell for exploring code
```

```
Sale Price
1         35546
250000     1252
200000     1232
150000     1189
175000     1118
          …
216756        1
70050         1
358590        1
215700        1
142515        1
Name: count, Length: 14720, dtype: int64
Max values in sale price: 71000000
Min values in sale price: 1
number of values between 1674 10000 and 15000
number of values under 37182 10000
Lowest 5 Sale Prices:
176008    1085
109682    1100
38829     1100
95766     1100
144757    1100
149691    1178
112997    1200
107580    1200
80321     1200
17808     1200
103859    1220
14355     1221
123254    1250
164798    1250
70382     1270
Name: Sale Price, dtype: int64
```

```
Highest 5 Sale Prices:
193635     7144704
117989     7400000
8043       7800000
194905     8000000
157521     8490078
19702     12000000
929       12250000
37835     12700000
71953     28000000
106057    71000000
Name: Sale Price, dtype: int64
```

0.0.6 Question 6a: Choose an additional feature

It's your turn to choose another feature to add to the model. Choose one new **quantitative** (not qualitative) feature and create Model 3 incorporating this feature (along with the features we've already chosen in Model 2). Try to choose a feature that will have a large impact on reducing the RMSE and/or will improve your residual plots. This can be a raw feature available in the dataset, or a transformation of one of the features in the dataset, or a new feature that you create from the dataset (see Project 1 for ideas).

Note: There is not one single right answer as to which feature to add, however **to receive credit on this question you should make sure the feature decreases the Cross Validation RMSE compared to Model 2 (i.e. we want to improve the model, not make it worse!)**

In the cell below, explain what additional feature you have chosen and why. Justify your reasoning. There are optional code cells provided below for you to use when exploring the dataset to determine which feature to add.

This problem will be graded based on your reasoning and explanation of the feature you choose, and then on your implementation of incorporating the feature.

NOTE Please don't add additional coding cells below or the Autograder will have issues. You do not need to use all the coding cells provided.

0.0.7 Question 6a Answer Cell:

In this cell, explain what feature you chose to add and why. Then give the equation for your new model (use the LaTeX from Model 2 from above and then add an additional term).

In [53]:
```
numeric_columns = tr_val_data.select_dtypes(include=['number'])
correlation_matrix = numeric_columns.corr()
correlation_matrix['Estimate (Building)']

# Show work in this cell exploring data to determine which feature to add
```

Out[53]:
```
PIN                   -0.378288
Property Class         0.304784
Neighborhood Code     -0.119656
Land Square Feet       0.163705
Town Code             -0.083422
Apartments             0.001153
Wall Material          0.110688
Roof Material          0.251133
Basement              -0.050316
Basement Finish       -0.133419
Central Heating        0.013642
```

```
Other Heating                0.043909
Central Air                  0.295400
Fireplaces                   0.513716
Attic Type                   0.055710
Attic Finish                -0.056069
Design Plan                  0.230424
Cathedral Ceiling            0.230571
Construction Quality        -0.193920
Site Desirability           -0.081020
Garage 1 Size                0.241888
Garage 1 Material            0.234590
Garage 1 Attachment         -0.071327
Garage 1 Area               -0.011244
Garage 2 Size               -0.017244
Garage 2 Material            0.029093
Garage 2 Attachment          0.020470
Garage 2 Area                0.011333
Porch                        0.032833
Other Improvements           0.021788
Building Square Feet         0.710719
Repair Condition            -0.125355
Multi Code                   0.063984
Number of Commercial Units   0.015058
Estimate (Land)              0.668045
Estimate (Building)          1.000000
Deed No.                     0.026045
Sale Price                   0.609286
Longitude                   -0.144528
Latitude                     0.393502
Census Tract                -0.105947
Multi Property Indicator     0.090462
Age                         -0.206364
Use                               NaN
O'Hare Noise                 0.012941
Floodplain                   0.016598
Road Proximity               0.033779
Sale Year                    0.026342
Sale Quarter                 0.027897
Sale Half-Year               0.027942
Sale Quarter of Year         0.011965
Sale Month of Year           0.011373
Sale Half of Year            0.013043
Most Recent Sale             0.075528
Age Decade                  -0.206364
Pure Market Filter           0.023185
Garage Indicator             0.083856
Neigborhood Code (mapping)  -0.119656
Town and Neighborhood       -0.151977
Lot Size                     0.163705
Name: Estimate (Building), dtype: float64
```

I am choosing the estimate of the estimated building price as it has a strong correlation with both the square footage of a property and the sale price of a properity. As we are trying to reduce the RMSE when we train

data using the information given we want information that is strongly correlated as that will allow us to make a better model for prediciting the cost of a property. It also make sense based on what it respresents as the data of the prices from last year should be a good predictor of the new prices of a properity and if a properity will be overvalued or undervalued. As we have been using the log of the square feet and sale price it seems resonable to use the log of the estimated value again in order to allow them to be compared at relativtly scales.

Model 3:

$$\text{Log Sale Price} = \theta_1(\text{Log Building Square Feet}) + \theta_2(\text{Shingle/Asphalt})$$

$$+\theta_3(\text{Tar\&Gravel}) + \theta_4(\text{Tile}) + \theta_5(\text{Shake}) + \theta_6(\text{Other}) + \theta_7(\text{Slate}) + \theta_8(\text{Estimate (Building)})$$

```
In [54]: tr_val_data['Estimate (Building)'].value_counts()
         tr_val_data[tr_val_data['Estimate (Building)'] > 0]

         # Optional code cell for additional work exploring data/ explaining which feature you chose.
```

Out[54]:

	PIN	Property Class	Neighborhood Code	Land Square Feet	\
0	17294100610000	203	50	2500.0	
1	13272240180000	202	120	3780.0	
2	25221150230000	202	210	4375.0	
3	10251130030000	203	220	4375.0	
4	31361040550000	202	120	8400.0	
…	…	…	…	…	
204787	25163010260000	202	321	4375.0	
204788	5063010090000	204	21	16509.0	
204789	16333020150000	202	90	3810.0	
204790	9242030500000	203	80	6650.0	
204791	19102030080000	203	30	2500.0	

	Town Code	Apartments	Wall Material	Roof Material	Basement	\
0	76	0.0	2.0	1.0	1.0	
1	71	0.0	2.0	1.0	1.0	
2	70	0.0	2.0	1.0	2.0	
3	17	0.0	3.0	1.0	1.0	
4	32	0.0	3.0	1.0	2.0	
…	…	…	…	…	…	
204787	72	0.0	2.0	1.0	1.0	
204788	23	0.0	1.0	1.0	1.0	
204789	15	0.0	2.0	1.0	1.0	
204790	22	0.0	2.0	1.0	1.0	
204791	72	0.0	1.0	1.0	1.0	

	Basement Finish	…	Sale Month of Year	Sale Half of Year	\
0	3.0	…	9	2	
1	1.0	…	5	1	
2	3.0	…	2	1	
3	3.0	…	7	2	
4	3.0	…	6	1	

```
…                 …  …                 …                 …
204787              1.0  …                 7                 2
204788              1.0  …                 3                 1
204789              1.0  …                 1                 1
204790              3.0  …                 2                 1
204791              3.0  …                 4                 1

        Most Recent Sale  Age Decade  Pure Market Filter  Garage Indicator  \
0                    1.0        13.2                   0               0.0
1                    1.0         9.6                   1               1.0
2                    0.0        11.2                   1               1.0
3                    1.0         6.3                   1               1.0
4                    0.0         6.3                   1               1.0
…                    …          …                     …               …
204787               0.0         5.8                   1               1.0
204788               1.0         9.3                   1               1.0
204789               1.0         5.9                   1               1.0
204790               1.0         6.0                   1               1.0
204791               0.0         4.7                   1               0.0

        Neigborhood Code (mapping)  Town and Neighborhood  \
0                               50                   7650
1                              120                  71120
2                              210                  70210
3                              220                  17220
4                              120                  32120
…                              …                    …
204787                         321                  72321
204788                          21                   2321
204789                          90                   1590
204790                          80                   2280
204791                          30                   7230

                                              Description  Lot Size
0       This property, sold on 09/14/2015, is a one-st…    2500.0
1       This property, sold on 05/23/2018, is a one-st…    3780.0
2       This property, sold on 02/18/2016, is a one-st…    4375.0
3       This property, sold on 07/23/2013, is a one-st…    4375.0
4       This property, sold on 06/10/2016, is a one-st…    8400.0
…                                                  …       …
204787  This property, sold on 07/23/2014, is a one-st…    4375.0
204788  This property, sold on 03/27/2019, is a one-st…   16509.0
204789  This property, sold on 01/31/2014, is a one-st…    3810.0
204790  This property, sold on 02/22/2018, is a one-st…    6650.0
204791  This property, sold on 04/22/2014, is a one-st…    2500.0

[203491 rows x 62 columns]
```

In [55]:
```
# Optional code cell for additional work exploring data/ explaining which feature you chose.
```

In [56]:
```
...
```

```
# Optional code cell for additional work exploring data/ explaining which feature you chose.
```

Out[56]: Ellipsis

0.0.8 Question 6b: Create Model 3

In the cells below fill in the code to create and analyze Model 3 (follow the Modeling steps outlined above).

PLEASE DO NOT ADD ANY ADDITIONAL CELLS IN THIS PROBLEM OR IT MIGHT MAKE THE AUTOGRADER FAIL

In [57]:
```
# Modeling Step 1:  Process the Data

# Hint: You can either use your implementation of the One Hot Encoding Function
#from Project Part 1, or use the staff's implementation

from feature_func import *

...
# Optional:  Define any helper functions you need for one-hot encoding above this line


def process_data_m3(df):

    data = df.copy()

    median_estimate = data["Estimate (Building)"][data["Estimate (Building)"] > 0].median()
    data["Estimate (Building)"] = data["Estimate (Building)"].replace(0, median_estimate)

    data["Log Sale Price"] = np.log(data["Sale Price"])

    data["Log Building Square Feet"] = np.log(data["Building Square Feet"])

    data["Log Estimate (Building)"] = np.log(data["Estimate (Building)"])

    data = ohe_roof_material(data)

    data = data[['Log Building Square Feet', 'Log Sale Price', 'Roof Material_1.0', 'Roof Mate

    return data



# Process the data for Model 3 (using the same tr and val datatsets we created in Question 3)
processed_train_m3 = process_data_m3(tr)

processed_val_m3 = process_data_m3(val)

# Create X (Dataframe) and y (series) to use to train the model
X_train_m3 = processed_train_m3.drop(columns=["Log Sale Price"])
y_train_m3 = processed_train_m3["Log Sale Price"]

X_valid_m3 = processed_val_m3.drop(columns=["Log Sale Price"])
y_valid_m3 = processed_val_m3["Log Sale Price"]
```

```
# Take a look at the result
display(X_train_m3.head())
display(y_train_m3.head())

display(X_valid_m3.head())
display(y_valid_m3.head())
```

```
        Log Building Square Feet  Roof Material_1.0  Roof Material_2.0  \
21302                   6.871091                1.0                0.0
19451                   7.576610                1.0                0.0
32018                   6.891626                1.0                0.0
144262                  7.186901                1.0                0.0
197227                  7.576610                1.0                0.0

        Roof Material_3.0  Roof Material_4.0  Roof Material_5.0  \
21302                 0.0                0.0                0.0
19451                 0.0                0.0                0.0
32018                 0.0                0.0                0.0
144262                0.0                0.0                0.0
197227                0.0                0.0                0.0

        Roof Material_6.0  Log Estimate (Building)
21302                 0.0                11.248048
19451                 0.0                11.510021
32018                 0.0                12.096425
144262                0.0                12.042259
197227                0.0                12.612637
```

```
21302     11.738466
19451     12.100712
32018     12.577291
144262    12.502467
197227    12.449019
Name: Log Sale Price, dtype: float64
```

```
        Log Building Square Feet  Roof Material_1.0  Roof Material_2.0  \
17112                   7.487174                1.0                0.0
189337                  7.688913                1.0                0.0
141725                  6.985642                1.0                0.0
9776                    6.846943                1.0                0.0
81676                   7.096721                1.0                0.0

        Roof Material_3.0  Roof Material_4.0  Roof Material_5.0  \
17112                 0.0                0.0                0.0
189337                0.0                0.0                0.0
141725                0.0                0.0                0.0
9776                  0.0                0.0                0.0
81676                 0.0                0.0                0.0
```

```
        Roof Material_6.0  Log Estimate (Building)
17112                 0.0                11.882211
189337                0.0                12.110112
141725                0.0                11.760410
9776                  0.0                11.569589
81676                 0.0                11.272763


17112     12.100707
189337    12.460715
141725    12.165251
9776      11.982929
81676     11.719940
Name: Log Sale Price, dtype: float64
```

In [58]:
```
# Modeling STEP 2:  Create and Fit a Multiple Linear Regression Model

model_m3 = lm.LinearRegression()

model_m3.fit(X_train_m3, y_train_m3)

# your code above this line to create and fit regression model for Model 3

y_predict_train_m3 = model_m3.predict(X_train_m3)

y_predict_valid_m3 = model_m3.predict(X_valid_m3)
```

In [59]:
```
# MODELING STEP 3:  Evaluate the RMSE for your model


# Training and validation errors for the model (in units dollars, not log(dollars))

training_error[2] = rmse(np.exp(y_train_m3), np.exp(y_predict_train_m3))
validation_error[2] = rmse(np.exp(y_valid_m3), np.exp(y_predict_valid_m3))


(print("3rd Model \nTraining RMSE: $ {}\nValidation RMSE: {}\n"
       .format(training_error[2], validation_error[2]))
)
```

```
3rd Model
Training RMSE: $ 189131.97010547665
Validation RMSE: 185859.5007674447
```

In [60]:
```
# MODELING STEP 4:  Conduct 5-fold cross validation for model and output RMSE
```

```
linear_model_m3_cv = lm.LinearRegression()


# Process the entire cleaned training_val dataset using the m3 pipeline
processed_full_m3 = process_data_m3(tr_val_clean)

# Split the processed_full_m3 Dataset into X and y to use in models.
X_full_m3 = processed_full_m3.drop(columns=["Log Sale Price"])
y_full_m3 = processed_full_m3["Log Sale Price"]


# Run cross_validate_rmse function:
cv_error_m3  = cross_validate_rmse(linear_model_m3_cv, X_full_m3, y_full_m3)

# Save the cross validation error for model 3 in our list to compare different models:

cv_error[2] = cv_error_m3

print("3rd Model Cross Validation RMSE: {}".format(cv_error[2]))
```

```
3rd Model Cross Validation RMSE: 188686.27102435727
```

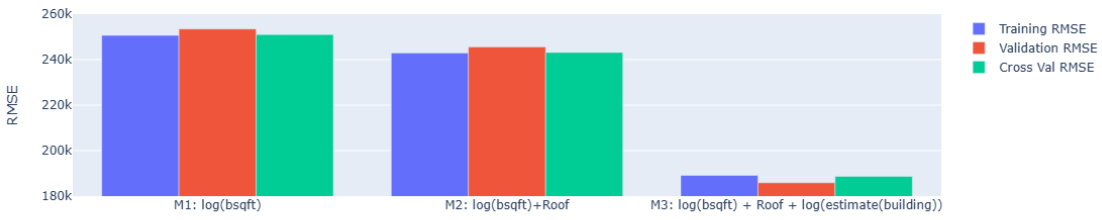
In [61]:
```
# MODELING STEP 5:  Add a name for your 3rd model describing the features
#and run this cell to Plot bar graph all 3 models

model_names[2] = "M3: log(bsqft) + Roof + log(estimate(building))"


fig = go.Figure([
go.Bar(x = model_names, y = training_error, name="Training RMSE"),
go.Bar(x = model_names, y = validation_error, name="Validation RMSE"),
go.Bar(x = model_names, y = cv_error, name="Cross Val RMSE")
])

fig.update_yaxes(range=[180000,260000], title="RMSE")

fig
```



In [62]:
```
# MODELING STEP 5 cont'd:  Plot 2 side-by-side residual plots
#(similar to Question 3, for validation data)

fig, ax = plt.subplots(1,2, figsize=(15, 5))

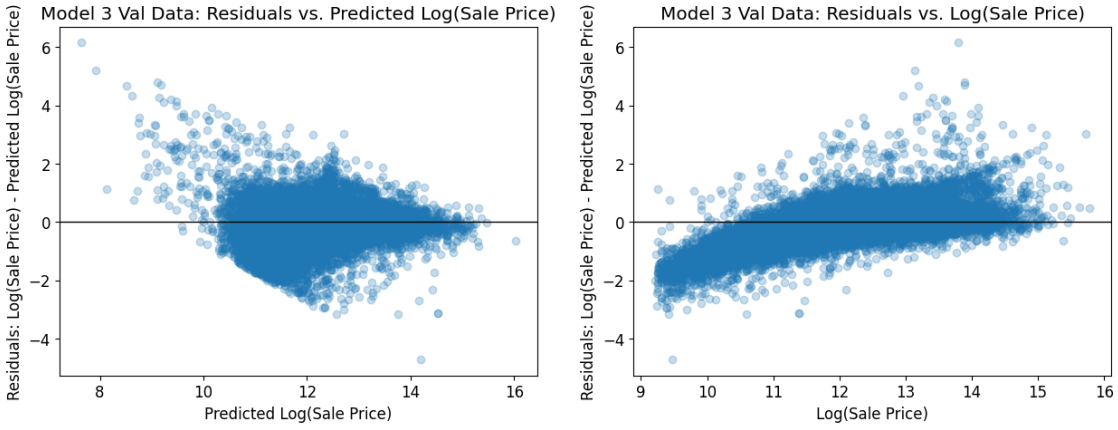

x_plt1 = y_predict_valid_m3
y_plt1 = y_valid_m3 - y_predict_valid_m3

x_plt2 = y_valid_m3
y_plt2 = y_valid_m3 - y_predict_valid_m3


ax[0].scatter(x_plt1, y_plt1, alpha=.25)
ax[0].axhline(0, c='black', linewidth=1)
ax[0].set_xlabel(r'Predicted Log(Sale Price)')
ax[0].set_ylabel(r'Residuals: Log(Sale Price) - Predicted Log(Sale Price)');
ax[0].set_title("Model 3 Val Data: Residuals vs. Predicted Log(Sale Price)")

ax[1].scatter(x_plt2, y_plt2, alpha=.25)
ax[1].axhline(0, c='black', linewidth=1)
ax[1].set_xlabel(r'Log(Sale Price)')
ax[1].set_ylabel(r'Residuals: Log(Sale Price) - Predicted Log(Sale Price)');
ax[1].set_title("Model 3 Val Data: Residuals vs. Log(Sale Price)")
```

Out[62]:
```
Text(0.5, 1.0, 'Model 3 Val Data: Residuals vs. Log(Sale Price)')
```



In [63]:
```
grader.check("q6b")
```

```
Out[63]: q6b results: All test cases passed!
```

0.0.9 Question 6c

- 6ci). Comment on your RMSE and residual plots from Model 3 compared to the first 2 models.
- 6cii). Are the residuals of your model still showing a trend that overestimates lower priced houses and underestimates higher priced houses? If so, how could you try to address this in the next round of modeling?
- 6ciii). If you had more time to improve your model, what would your next steps be?

6ci) The RMSE of the model is much smaller than that of the first 2 models and the residuals ar a lot more tightly and unifromly packed around zero meaning that the model's predictions are a lot more consistent when compared against the first 2 models. We still do see that the validation data still has a trend just as it did in the first 2 models meaning there is still ways to improve our model.

6cii)There is still a trend which overestimates the lower valued priced house and the underestimate higher prices house. One way this could be address it by adding another variable or changing the scale we are looking at each of the variables so that we can transform the residuals from being over and underestamated.

6ciii) I would like to try another transformation of the sale price and adding another interation term which would better capture the trend of the model.

0.1 Question 7a

When evaluating your model, we used RMSE. In the context of estimating the value of houses, what does the residual mean for an individual homeowner? How does it affect them in terms of property taxes? Discuss the cases where residual is positive and negative separately.

A positive residual means that the model is underestimating the value of their house which mean that they would play less in property taxes. In the case of negative residuals that would mean that a homeowner's hosue would be overvalued and they are paying more property taxes than they should be.

0.2 Question 7b

Reflecting back on your exploration in Questions 5 and 6a, in your own words, what makes a model's predictions of property values for tax assessment purposes "fair"?

This question is open-ended and part of your answer may depend upon your specific model; we are looking for thoughtfulness and engagement with the material, not correctness.

Hint: Some guiding questions to reflect on as you answer the question above: What is the relationship between RMSE, accuracy, and fairness as you have defined it? Is a model with a low RMSE necessarily accurate? Is a model with a low RMSE necessarily "fair"? Is there any difference between your answers to the previous two questions? And if so, why?

The most fair model would be on which has a low RMSE however as the accuracy would like suffer due to overfitting. The model would also need to address the systemic basis that is inherent with the fact that many of these lower value houses represent a disprapotionate amount of mintorites which results in basis and unfairness. The difference is a result of even an accurate model not being necessarily fair as if the model is prediciting trends that benefit one group over the other inherently then an accurate model could be unfair.

0.3 Extra Credit Step 1: Creating Your Model

Complete the modeling steps (you can skip the cross validation step to save memory) in the cells below.

DO NOT ADD ANY EXTRA CELLS BELOW (for this part of the problem)

1 Please include all of your feature engineering processes

```
# for the training, validation and test sets.
# dataset_type is a flag to use as follows:
# dataset_type=1  imples the data is the training data
# dataset_type=2 implies the data is the validation data
# dataset_type = 3 imples the data is the test data

#Important Instructions:
# When processing the training data, you CAN drop any rows/data that you deem to be outliers
# When processing the validation data you CANNOT drop any rows
# When processing the test data, you CANNOT drop any rows, and you CANNOT reference "Sale Price", as it
```

In [84]:
```
# Modeling Step 1:  Process the Data


        # Hint: You can either use your implementation of the One Hot Encoding Function from
        #Project Part 1, or use the staff's implementation
        from feature_func import *


        ...

        # # Optional:  Define any helper functions you need (for example, for one-hot
        # #encoding, etc) above this line


        def process_data_ec(df, dataset_type):

            """
            Function that includes all feature engineering processes
            for the training, validation and test sets for your extra credit model.

            Important Instructions:
            When processing the training data, you CAN drop any rows/data that you deem to be outliers
            When processing the validation data you CANNOT drop any rows (even what you deem to be an
            When processing the test data, you CANNOT drop any rows, and you CANNOT reference "Sale Pr

            dataset_type is a flag to use as follows:
                dataset_type=1  implies the data is the training data
                dataset_type=2 implies the data is the validation data
```

```
        dataset_type = 3 imples the data is the test data

    """


    data = df.copy()

    median_estimate = data["Estimate (Building)"][data["Estimate (Building)"] > 0].median()
    data["Estimate (Building)"] = data["Estimate (Building)"].replace(0, median_estimate)

    data["Log Building Square Feet"] = np.log(data["Building Square Feet"])

    data["Log Estimate (Building)"] = np.log(data["Estimate (Building)"])

    data = ohe_roof_material(data)

    if dataset_type < 3:
        data["Log Sale Price"] = np.log(data["Sale Price"])
        data = data[['Log Building Square Feet', 'Log Estimate (Building)', 'Log Sale Price',
                 'Roof Material_1.0', 'Roof Material_2.0', 'Roof Material_3.0',
                 'Roof Material_4.0', 'Roof Material_5.0', 'Roof Material_6.0', ]]
    else:
        data = data[['Log Building Square Feet', 'Log Estimate (Building)',
                 'Roof Material_1.0', 'Roof Material_2.0', 'Roof Material_3.0',
                 'Roof Material_4.0', 'Roof Material_5.0', 'Roof Material_6.0',
                ]]
    return data


# Use the same original train and valid datasets from 3a (otherwise the
# validation errors aren't comparable).  Don't resplit the data.

# Process the data
processed_train_ec = process_data_ec(tr, dataset_type=1)

processed_val_ec = process_data_ec(val, dataset_type=2)


X_train_ec = processed_train_ec.drop(columns=["Log Sale Price"])
y_train_ec = processed_train_ec["Log Sale Price"]

X_valid_ec = processed_val_ec.drop(columns=["Log Sale Price"])
y_valid_ec =  processed_val_ec["Log Sale Price"]



# Take a look at the result
display(X_train_ec.head())
display(y_train_ec.head())

display(X_valid_ec.head())
display(y_valid_ec.head())
```

```
       Log Building Square Feet  Log Estimate (Building)  Roof Material_1.0  \
21302                  6.871091                11.248048                1.0
19451                  7.576610                11.510021                1.0
32018                  6.891626                12.096425                1.0
144262                 7.186901                12.042259                1.0
197227                 7.576610                12.612637                1.0

       Roof Material_2.0  Roof Material_3.0  Roof Material_4.0  \
21302                0.0                0.0                0.0
19451                0.0                0.0                0.0
32018                0.0                0.0                0.0
144262               0.0                0.0                0.0
197227               0.0                0.0                0.0

       Roof Material_5.0  Roof Material_6.0
21302                0.0                0.0
19451                0.0                0.0
32018                0.0                0.0
144262               0.0                0.0
197227               0.0                0.0


21302     11.738466
19451     12.100712
32018     12.577291
144262    12.502467
197227    12.449019
Name: Log Sale Price, dtype: float64


       Log Building Square Feet  Log Estimate (Building)  Roof Material_1.0  \
17112                  7.487174                11.882211                1.0
189337                 7.688913                12.110112                1.0
141725                 6.985642                11.760410                1.0
9776                   6.846943                11.569589                1.0
81676                  7.096721                11.272763                1.0

       Roof Material_2.0  Roof Material_3.0  Roof Material_4.0  \
17112                0.0                0.0                0.0
189337               0.0                0.0                0.0
141725               0.0                0.0                0.0
9776                 0.0                0.0                0.0
81676                0.0                0.0                0.0

       Roof Material_5.0  Roof Material_6.0
17112                0.0                0.0
189337               0.0                0.0
141725               0.0                0.0
9776                 0.0                0.0
81676                0.0                0.0


17112     12.100707
189337    12.460715
```

```
141725    12.165251
9776      11.982929
81676     11.719940
Name: Log Sale Price, dtype: float64
```

In [85]:
```
# Run this code to make sure you haven't dropped any of the rows in the validation set

assert X_valid_ec.shape[0] == 33475
```

In [86]:
```
# Modeling STEP 2:  Create a Multiple Linear Regression Model


linear_model_ec = lm.LinearRegression()

linear_model_ec.fit(X_train_ec, y_train_ec)


# your code above this line to create regression model

y_predict_train_ec = linear_model_ec.predict(X_train_ec)

y_predict_valid_ec = linear_model_ec.predict(X_valid_ec)
```

In [87]:
```
# MODELING STEP 3:  Evaluate the RMSE for your model


# Training and test errors for the model
#(in its original values before the log transform)

training_error_ec = rmse(np.exp(y_train_ec), np.exp(y_predict_train_ec))
validation_error_ec = rmse(np.exp(y_valid_ec), np.exp(y_predict_valid_ec))


(print("Extra Credit \nTraining RMSE:$ {}\nValidation RMSE:$ {}\n"
       .format(training_error_ec, validation_error_ec))
)
```

```
Extra Credit
Training RMSE:$ 189131.9701054766
Validation RMSE:$ 185859.50076744476
```

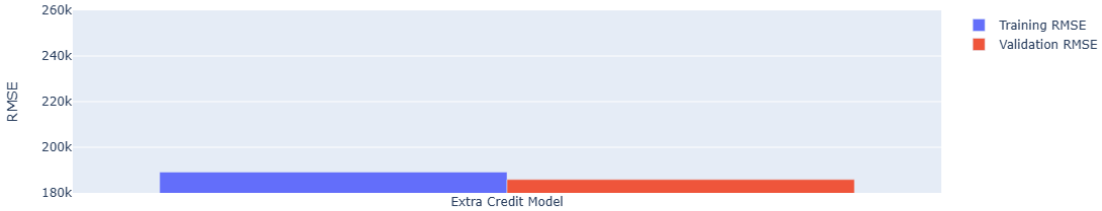
In [89]:
```
# Optional: Run this cell to visualize

#import plotly.graph_objects as go
```

```
fig = go.Figure([
go.Bar(x = ["Extra Credit Model"], y = [training_error_ec], name="Training RMSE"),
go.Bar(x = ["Extra Credit Model"], y = [validation_error_ec], name="Validation RMSE"),

])


fig
fig.update_yaxes(range=[180000,260000], title="RMSE")
# Feel free to update the range as needed
```



In [92]:
```
# MODELING STEP 5: Plot 2 side-by-side residual plots for validation data

fig, ax = plt.subplots(1,2, figsize=(15, 5))

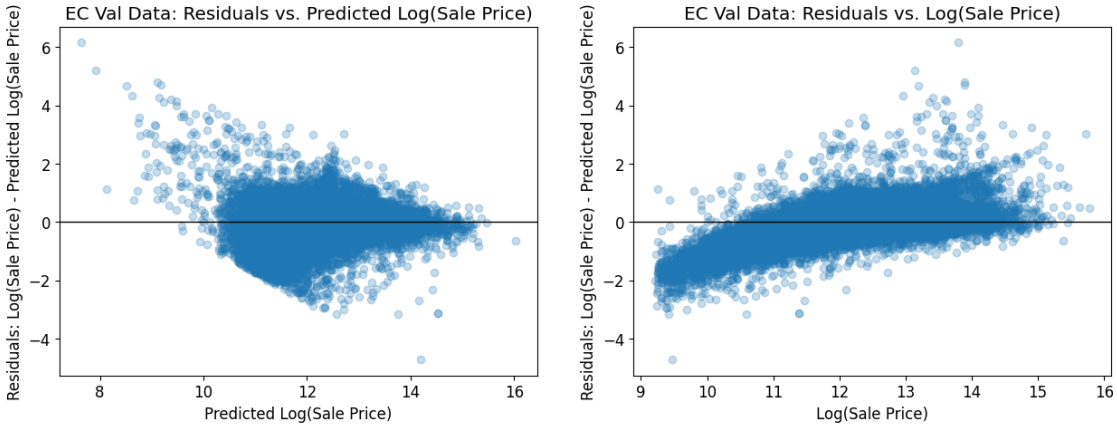

x_plt1 = y_predict_valid_ec
y_plt1 = y_valid_ec - y_predict_valid_ec

x_plt2 = y_valid_ec
y_plt2 = y_valid_ec - y_predict_valid_ec


ax[0].scatter(x_plt1, y_plt1, alpha=.25)
ax[0].axhline(0, c='black', linewidth=1)
ax[0].set_xlabel(r'Predicted Log(Sale Price)')
ax[0].set_ylabel(r'Residuals: Log(Sale Price) - Predicted Log(Sale Price)');
ax[0].set_title("EC Val Data: Residuals vs. Predicted Log(Sale Price)")

ax[1].scatter(x_plt2, y_plt2, alpha=.25)
ax[1].axhline(0, c='black', linewidth=1)
ax[1].set_xlabel(r'Log(Sale Price)')
ax[1].set_ylabel(r'Residuals: Log(Sale Price) - Predicted Log(Sale Price)');
ax[1].set_title("EC Val Data: Residuals vs. Log(Sale Price)")
```

Out[92]: Text(0.5, 1.0, 'EC Val Data: Residuals vs. Log(Sale Price)')



1.1 Extra Credit Step 2: Explanation (Required for points on model above):

- Explain what you did to create your model. What versions did you try? What worked and what didn't?
- Comment on the RMSE and residual plots from your model. Are the residuals of your model still showing a trend that overestimates lower priced houses and underestimates higher priced houses?

Write your answers in the text cell below

This was the model that I used for part 6. I did try doing transformation of the sale price in order to better fit the line so that the bulding of the data towards the top left dissapereared but every attempt would increas the RMSE. I didn't attempt to add another variable as I couldn't find one by looking through the data the would feel